

Progress Report – Week 7

Học phần: Operating Systems [Multimedia] – 20252

Nguyễn Sỹ Tuấn

Lê Huy Sơn

Vũ Tiến Long

Giai đoạn báo cáo: 9/5/2026 – 15/5/2026

Mã số dự án: 25236
Tên dự án: Optimize pipe implementation to improve IPC throughput
Thiết kế: Link Figma
Quản lý tác vụ: Link JIRA
Thành viên: Nguyễn Sỹ Tuấn, Lê Huy Sơn, Vũ Tiến Long

1 Summary of latest progress

(Viết khoảng 200–300 từ để tóm tắt quá trình thực hiện trong giai đoạn này và tổng mức độ hoàn thành hiện tại.)

Đây là tuần cuối cùng của dự án tối ưu hóa pipe. Căn cứ kế hoạch đề ra ở cuối tuần 6, chúng tôi đặt ra các mục tiêu:

1. Triển khai **v6 – Priority Boost**: scheduler ưu tiên tiến trình vừa được wakeup.
2. Triển khai **v7 – Cache-line Alignment**: tách `nread/nwrite` lên hai cache line khác nhau.
3. Benchmark đầy đủ tất cả 7 phiên bản và vẽ biểu đồ tổng thể.

Chúng tôi đã hoàn thành cả ba mục tiêu trên và mở rộng thêm hai công việc ngoài kế hoạch:

- **v6 – Priority Boost**: Thêm trường `priority` vào `struct proc`, hàm `wakeup()` gán `priority = PRIORITY_BOOST = 10` cho process được đánh thức. Scheduler luôn chọn process có `priority` cao nhất trong số các `RUNNABLE` process. Sau mỗi tick, `yield()` giảm `priority` 1 đơn vị, tránh starvation. Kết quả: **13,981,013 B/s** tại `chunk_bytes = 4096` – tăng **1.33×** so với v5.
- **v7 – Cache-line Alignment**: Chèn `char _pad_w[60]` và `char _pad_r[60]` giữa `nwrite` và `nread` trong `struct pipe`, đặt hai biến lên hai cache line 64-byte riêng biệt, loại bỏ false sharing trong môi trường multi-core. Trên single-CPU, v7 hơi chậm hơn v6 (–14%) vì `struct` lớn hơn – đây là kết quả đúng và có thể giải thích.
- **v8 – Newpipe (tổng hợp tất cả cơ chế)**: Kết hợp 6 cơ chế tối ưu (bulk copy, 2-trang buffer 8 KB, page-indexed addressing, lazy wakeup, priority boost, cache-line alignment) vào một phiên bản hoàn chỉnh. Kết quả: **20,971,520 B/s** tại `chunk_bytes ≥ 2048` – **36.5×** so với v1 baseline.
- **IPC use-case programs**: Viết 5 chương trình user-space minh họa các IPC pattern phổ biến: `pipe_wc`, `pipe_chain`, `pipe_fanout`, `pipe_pingpong`, và `pipe_live` (hiển thị tốc độ real-time mỗi tick).

Tổng mức độ hoàn thành: 100% – toàn bộ 8 phiên bản (v1–v8) đã triển khai, benchmark, phân tích và tài liệu hóa đầy đủ.

2 Review of technical activities

2.1 Activity 1: v6 – Priority Boost

2.1.1 Bottleneck nhắm tới

Sau v5, throughput đạt ~10.5 MB/s tại `chunk_bytes = 4096` nhưng vẫn còn một nguồn overhead quan trọng: **scheduler delay**. Khi writer gọi `wakeup()` để đánh thức reader, reader chuyển sang trạng thái `RUNNABLE` nhưng không được scheduler chọn chạy ngay – nó phải xếp hàng và chờ đến lượt trong round-robin. Khoảng thời gian này, CPU nhàn rỗi hoặc chạy các process khác trong khi pipe data đang chờ được xử lý.

Lý thuyết liên quan – Scheduling và IO-bound boosting:

Trong lý thuyết scheduling, tiến trình được phân loại thành hai loại:

- **CPU-bound:** chạy lâu liên tục, hiếm khi block – ví dụ: tính toán số học.
- **IO-bound (interactive):** thường xuyên block chờ I/O, mỗi lần chạy ngắn – ví dụ: đọc pipe, đọc file.

Thuật toán **Round-Robin** của xv6 gốc không phân biệt hai loại này – mọi `RUNNABLE` process được chọn tuần tự. Điều này gây **convoy effect**: một IO-bound process (reader) vừa được wakeup phải chờ nhiều CPU-bound process khác chạy hết quantum trước, làm tăng latency mỗi hop IPC.

Multi-Level Feedback Queue (MLFQ) – Corbató 1962 – giải quyết vấn đề này bằng cách ưu tiên IO-bound process: khi một process block chờ I/O, nó được thăng lên queue ưu tiên cao. v6 áp dụng triết lý MLFQ ở dạng đơn giản: **priority boost** cho process vừa bị wakeup từ sleep trên pipe.

2.1.2 Thay đổi trong `struct proc` và scheduler

Thêm trường `priority` vào `struct proc` (`kernel/proc.h`):

```
#if PIPE_VERSION >= 6
#define PRIORITY_BASE 0
#define PRIORITY_BOOST 10

struct proc {
    // ... các trường hiện có ...
    int priority;    // scheduling priority (0 = bình thường, 10 = boost)
};
#endif
```

Hàm `wakeup()` – gán boost khi đánh thức:

```
void wakeup(void *chan) {
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()) {
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                p->state = RUNNABLE;
                #if PIPE_VERSION >= 6
                p->priority = PRIORITY_BOOST;    // dat priority cao nhất
                #endif
            }
            release(&p->lock);
        }
    }
}
```

Scheduler – chọn process có priority cao nhất:

```
void scheduler(void) {
    struct cpu *c = mycpu();
    for(;;) {
        intr_on();
        struct proc *chosen = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
#if PIPE_VERSION >= 6
                if(chosen == 0 || p->priority > chosen->priority)
                    chosen = p;
#else
                chosen = p; // round-robin goc
#endif
            }
            if(p != chosen) release(&p->lock);
        }
        if(chosen) {
            chosen->state = RUNNING;
            c->proc = chosen;
            swtch(&c->context, &chosen->context);
            release(&chosen->lock);
        }
    }
}
```

yield() – decay priority mỗi tick để tránh starvation:

```
void yield(void) {
    struct proc *p = myproc();
    acquire(&p->lock);
#if PIPE_VERSION >= 6
    if(p->priority > PRIORITY_BASE)
        p->priority--; // giảm 1 đơn vị mỗi khi yield
#endif
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}
```

2.1.3 Sơ đồ trước và sau Priority Boost

Trước v6 (round-robin) – reader phải chờ:

Tick timeline (mỗi o = 1 quantum):

CPU: [P_other][P_other][P_other][READER][WRITER][READER][WRITER]

Reader được wakeup nhưng phải chờ 3 quantum của các process khác trước khi được scheduler chọn.

Sau v6 (priority boost) – reader chạy NGAY:

CPU: [P_other][READER][WRITER][READER][WRITER]

reader vừa được wakeup với priority=10, scheduler chọn ngay tick tiếp theo. P_other phải nhường.

priority decay: 10 → 9 → 8 → ... → 0 (sau 10 tick → bình thường)

Số lần context switch v5 vs v6 (tính cho 4 MB, chunk=1024):

```
v5: ~2,048 cap sleep/wakeup x ~3 tick cho scheduler = ~6,144 quantum lang phi
v6: ~2,048 cap sleep/wakeup x ~1 tick cho (boost)    = ~2,048 quantum lang phi
→ Giảm ~3x scheduler delay overhead
```

2.1.4 Kết quả đo đặc v6

chunk_bytes	v5 (B/s)	v6 (B/s)	Tăng
64	1,048,576	1,061,849	1.01×
128	1,906,501	1,997,287	1.05×
256	3,355,443	3,495,253	1.04×
512	5,242,880	5,592,405	1.07×
1024	6,990,506	8,388,608	1.20×
2048	9,320,675	10,485,760	1.12×
4096	10,485,760	13,981,013	1.33×

Phân tích: Speedup lớn nhất ở chunk_bytes lớn (1024–4096) vì đây là kích thước mà số lần sleep/wakeup nhiều nhất và scheduler delay ảnh hưởng nhiều nhất. Ở chunk_bytes = 64, pipe hiếm khi đầy nên reader ít khi ngủ → boost có ít cơ hội phát huy.

2.2 Activity 2: v7 – Cache-line Alignment

2.2.1 Bottleneck nhắm tới – False Sharing

Trong struct pipe của v1–v6, nwrite và nread nằm cạnh nhau trong memory, thường trên cùng một cache line 64 byte. Trong môi trường multi-core (CPUS ≥ 2), điều này gây false sharing:

- Core 0 (writer) ghi nwrite → cache line bị đánh dấu **Modified** (MESI: M).
- Core 1 (reader) muốn đọc nread → cache line đó đang ở Core 0 → phải **invalidate** cache line của Core 1 → cache miss → stall.

Hai biến nread và nwrite **không chia sẻ dữ liệu thật**, nhưng vì cùng cache line → CPU phải synchronize giữa hai core – đây là “false” sharing, tốn kém như real sharing nhưng không có lý do.

Lý thuyết liên quan – MESI Protocol và False Sharing:

MESI là giao thức coherence phổ biến nhất trong CPU multi-core:

Trạng thái	Ý nghĩa
M (Modified)	Core này có bản sao mới nhất, các core khác không có
E (Exclusive)	Core này có bản sao, các core khác không có
S (Shared)	Nhiều core có bản sao giống nhau
I (Invalid)	Cache line bị vô hiệu, phải reload từ memory/L3

False sharing: hai core ghi vào hai biến khác nhau nhưng cùng cache line:

```
Core 0 ghi nwrite:  cache line → M (Core 0) → I (Core 1)
Core 1 đọc nread:  cache miss! phải fetch lại từ L3/LLC
Core 0 ghi nwrite:  cache line → M (Core 0) → I (Core 1)
Core 1 đọc nread:  cache miss! lại fetch lại...
```

Mỗi cycle false sharing: ~100–300 cycle stall. **Giải pháp chuẩn trong Linux kernel:** macro `__cacheline_aligned` chèn padding để biến bắt đầu tại địa chỉ bội số của 64.

2.2.2 Cấu trúc struct pipe trước và sau v7

Trước v7 – nwrite và nread cùng cache line:

```
Offset 0:  spinlock  (40 B)  +
Offset 40:  char *data ( 8 B) |  cache line 0 (byte 0--63)
Offset 48:  nread    ( 4 B) |
Offset 52:  nwrite   ( 4 B) |
Offset 56:  readopen ( 4 B) |
Offset 60:  writeopen ( 4 B) +
                                     ← nread va nwrite cung cache line!
Core 0 ghi nwrite → invalidate cache line → Core 1 miss nread
```

Sau v7 – padding tách nwrite và nread:

```
#if PIPE_VERSION >= 7
struct pipe {
    struct spinlock lock;           // 40 B -- cache line 0
    char *data;                    // 8 B
    int readopen;                  // 4 B
    int writeopen;                 // 4 B
    uint nwrite;                   // 4 B +
    char _pad_w[60];               // 60 B |  cache line 1 (byte 64--127)
    uint nread;                    // 4 B + → day nread sang cache line 2
    char _pad_r[60];               // 60 B -- cache line 2 (byte 128--191)
    int need_wakeup;               // 4 B -- cache line 3
    int was_full;                  // 4 B
};
#endif
```

Layout sau v7:

```
cache line 0 (byte 0-- 63):  spinlock + data ptr + readopen + writeopen + nwrite
cache line 1 (byte 64--127):  _pad_w (padding)
cache line 2 (byte 128--191):  nread + _pad_r (padding)
cache line 3 (byte 192--255):  flags lazy wakeup

Core 0 ghi nwrite → chi dirty cache line 0 → Core 1 KHÔNG bị invalidate
Core 1 doc nread → cache line 2 van valid → KHÔNG có stall
```

2.2.3 Nhận xét về kết quả đo đạc v7 trên single-CPU

chunk_bytes	v6 (B/s)	v7 (B/s)	Thay đổi
64	1,061,849	1,075,462	+1.3%
128	1,997,287	1,997,287	0%
256	3,495,253	3,495,253	0%
512	5,592,405	5,592,405	0%
1024	8,388,608	8,388,608	0%
2048	10,485,760	10,485,760	0%
4096	13,981,013	11,983,725	-14%

Phân tích – tại sao v7 hơi chậm hơn v6 ở chunk=4096?

Kết quả này đúng và có thể giải thích được trên single-CPU (CPUS=1):

1. **False sharing không xảy ra trên 1 core** – không có Core 1 để bị invalidate, nên lợi ích của alignment = 0.
2. **Struct lớn hơn đáng kể** – từ ~72 byte lên ~256 byte (padding 2×60 byte). Struct lớn làm tăng số cache line phải load trong hot path.

3. Lợi ích thực sự của v7 chỉ thấy được với $\text{CPUS} \geq 2$ – v7 là tối ưu hóa cho scalability, không phải throughput đơn luồng.

2.3 Activity 3: v8 – Newpipe (tổng hợp tất cả cơ chế)

2.3.1 Mục tiêu – Kết hợp 6 cơ chế thành 1 phiên bản hoàn chỉnh

Sau khi phân tích 7 phiên bản, chúng tôi xác định 6 cơ chế tối ưu không xung đột nhau và có thể kết hợp:

#	Cơ chế	Nguồn gốc	Bottleneck giải quyết
1	Bulk copyin/copyout	v2	n lần page walk $\rightarrow$ 2 lần/write
2	2-trang buffer (8 KB)	v4 mở rộng	capacity 4 KB $\rightarrow$ 8 KB
3	Page-indexed addressing	mới trong v8	tránh cross-page read
4	Lazy wakeup	v5	$O(\text{NPROC})$ wakeup $\rightarrow O(1)$
5	Priority boost	v6	scheduler delay sau wakeup
6	Cache-line alignment	v7	false sharing trên multi-core

2.3.2 Cơ chế mới trong v8 – Page-indexed Addressing

v4 dùng 1 trang (4096 byte) với flat ring – memmove có thể bị cắt ở ranh giới trang khi ring wrap-around. v8 dùng **2 trang riêng biệt** và đảm bảo mỗi copyin/copyout chỉ nằm trong 1 trang:

```
#define DATA_PAGES 2
#define PIPE_TOTAL (DATA_PAGES * PGSIZE) // 8192 byte

struct pipe {
    struct spinlock lock;
    char *data[DATA_PAGES]; // 2 con trỏ tới 2 trang riêng biệt
    uint nwrite;
    char _pad_w[60]; // cache-line padding
    uint nread;
    char _pad_r[60];
    int need_wakeup; // lazy wakeup flag (writer side)
    int was_full; // lazy wakeup flag (reader side)
};
```

Page-indexed addressing trong pipewrite:

```
uint pos = pi->nwrite % PIPE_TOTAL; // vị trí tuyệt đối trong 8KB ring
uint page_idx = pos / PGSIZE; // thuộc trang 0 hay trang 1?
uint page_off = pos % PGSIZE; // offset trong trang đó
uint till_page = PGSIZE - page_off; // còn bao nhiêu byte đến cuối trang

int to_copy = umin(n - i, umin(available, till_page));

// copyin KHÔNG BAO GIỜ vượt ranh giới trang
copyin(pr->pagetable, pi->data[page_idx] + page_off, addr + i, to_copy);
pi->nwrite += to_copy;
```

Sơ đồ 2-trang ring buffer của v8:

pi->data[0] (trang 1, PGSIZE=4096 B)	pi->data[1] (trang 2, 4096 B)
<pre>+-----+ [0..4095] đã đọc... valid data +-----+</pre>	<pre>+-----+ [0..4095] valid data trong..... +-----+</pre>
^ nread % 8192 = 1500 (trang 0, offset 1500)	^ nwrite % 8192 = 5500 (trang 1, offset 1404)

Lan ghi tiep: page_idx=1, page_off=1404, till_page=2692
→ copyin(data[i]+1404, src, 2692) -- nam gon trong trang 1, khong cross-page

OOM-safe rollback trong pipealloc:

```
struct pipe *pi = (struct pipe*) kalloc(); // trang struct
if(pi == 0) goto bad;

int i;
for(i = 0; i < DATA_PAGES; i++){
    pi->data[i] = kalloc(); // trang data
    if(pi->data[i] == 0){
        int j;
        for(j = 0; j < i; j++){
            kfree(pi->data[j]); // rollback cac trang da cap
        }
        kfree((void*)pi);
        goto bad;
    }
}
```

2.3.3 Tổng hợp 6 cơ chế và tác động lý thuyết

Bottleneck 1: Per-byte copy overhead
v1: 4,194,304 lan copyin (4MB / 1 byte)
v8: 2,048 lan copyin (4MB / 2048 B chunk trung binh)
→ Giam 2,048x so lan page table walk

Bottleneck 2: Sleep/wakeup frequency
v1: ~16,384 lan sleep/wakeup (512B buffer x 4MB = 8192 lan day)
v8: ~1,024 lan sleep/wakeup (8KB buffer x 4MB = 512 lan day)
→ Giam 16x so lan context switch

Bottleneck 3: Wakeup scan overhead
v1: 0(NPROC=64) moi wakeup, luc nao cung goi
v8: 0(1) fast path khi peer khong ngu (lazy wakeup)
→ Giam ~64x overhead wakeup trong fast path

Bottleneck 4: Scheduler delay sau wakeup
v1: reader cho round-robin (~3 quantum)
v8: reader duoc boost priority → chay tick tiep theo
→ Giam ~3x latency sau wakeup

Bottleneck 5: False sharing (multi-core)
v1: nwrite/nread cung cache line → MESI invalidate lien tuc
v8: padding 60B tach biet → khong co false sharing
→ Loai bo hoan toan tren multi-core

2.3.4 Kết quả đo đạc v8 và so sánh toàn bộ v1-v8

chunk	v1	v5	v6	v7	v8 (B/s)	v8/v1
64	206,108	1,048,576	1,061,849	1,075,462	1,075,462	5.2×
128	307,275	1,906,501	1,997,287	1,997,287	2,097,152	6.8×
256	427,990	3,355,443	3,495,253	3,495,253	3,813,003	8.9×
512	524,288	5,242,880	5,592,405	5,592,405	6,990,506	13.3×
1024	551,882	6,990,506	8,388,608	8,388,608	13,981,013	25.3×
2048	562,993	9,320,675	10,485,760	10,485,760	20,971,520	37.2×
4096	574,562	10,485,760	13,981,013	11,983,725	20,971,520	36.5×

Biểu đồ ASCII throughput tại chunk_bytes = 1024 (B/s):

	0	3M	6M	9M	12M	15M	18M	21M	B/s
		-----+	-----+	-----+	-----+	-----+	-----+	-----+	
v1	#								551,882
v2	###								1,582,756
v3	#####								3,355,443
v4	#####								3,994,575
v5	#####								6,990,506
v6	#####								8,388,608
v7	#####								8,388,608
v8	#####								13,981,013

Biểu đồ ASCII throughput tại chunk_bytes = 4096 (B/s):

	0	5M	10M	15M	20M	B/s
		-----+	-----+	-----+	-----+	
v1	#					574,562
v2	####					1,864,135
v3	#####					6,452,775
v4	#####					10,485,760
v5	#####					10,485,760
v6	#####					13,981,013
v7	#####					11,983,725
v8	#####					20,971,520

2.4 Activity 4: IPC Use-case Programs

Để chứng minh tính ứng dụng thực tế của các cơ chế tối ưu, chúng tôi viết 5 chương trình user-space minh họa các IPC pattern phổ biến.

2.4.1 Tổng quan 5 chương trình

File	IPC Pattern	Pipe	Proc	Mô tả
pipe_wc.c	Producer-Consumer	1	2	gen_text wc
pipe_chain.c	Pipeline (dây chuyền)	2	3	gen XOR verify
pipe_fanout.c	Fan-out / Broadcast	N	N+1	1 writer → 3 readers
pipe_pingpong.c	Request-Response (RPC)	2	2	10000 round-trip, đo RTT
pipe_live.c	Live monitor	1	2	KB/s mỗi tick (100ms)

2.4.2 Kết quả benchmark IPC use-cases – v1 vs v8

```

pipe_wc -- Streaming text word-count:
v1: elapsed 80 ticks (8000 ms), throughput 524,288 B/s ( 512 KB/s)
v8: elapsed 3 ticks (300 ms), throughput 13,981,013 B/s (13,653 KB/s) ~27x

pipe_chain -- 3-stage XOR transform pipeline:
v1: elapsed 152 ticks (15200 ms), throughput 275,941 B/s ( 269 KB/s)
v8: elapsed 5 ticks (500 ms), throughput 8,388,608 B/s (8,192 KB/s) ~30x

pipe_fanout -- Fan-out 1 writer → 3 readers:
v1: elapsed 113 ticks (11300 ms), throughput 556,766 B/s ( 543 KB/s)
v8: elapsed 2 ticks (200 ms), throughput 31,457,280 B/s (30,720 KB/s) ~57x

pipe_pingpong -- Bidirectional round-trip IPC:
v1: elapsed 180 ticks (18000 ms), ops/sec = 555, latency = 1.80 ms/rtt
v8: elapsed 29 ticks (2900 ms), ops/sec = 3448, latency = 0.29 ms/rtt ~6x

```


Use case	v1	v8	Speedup	Cơ chế chủ đạo
pipe_wc	512 KB/s	13,653 KB/s	~27×	bulk copy + 8KB buffer
pipe_chain	269 KB/s	8,192 KB/s	~30×	2 pipe × bulk copy
pipe_fanout	543 KB/s	30,720 KB/s	~57×	8KB buffer × 3 pipe
pipe_pingpong	555 ops/s	3,448 ops/s	~6×	priority boost (latency)

Nhận xét: `pipe_fanout` đạt speedup cao nhất (~57×) vì với 3 pipe song song và buffer 8 KB mỗi pipe, writer gần như không bao giờ block. `pipe_pingpong` speedup thấp hơn (~6×) vì bị giới hạn bởi latency: mỗi round-trip cần đúng 2 lần wakeup không thể tránh khỏi.

2.4.3 `pipe_live` – Hiển thị tốc độ real-time mỗi 100ms

`pipe_live` in một dòng mỗi tick (100ms) hiển thị tốc độ hiện tại và thanh tiến trình, cho phép quan sát trực tiếp dữ liệu đang chảy qua pipe giữa hai tiến trình. Kiến trúc: `fork()` tạo hai tiến trình – Parent (WRITER) ghi liên tục, Child (READER) đọc và in live stats.

Output mẫu v8 (16 MB):

```
WRITER [PID 3] --[pipe]--> READER [child]

tick | progress                                | pct | KB/s
-----|-----|-----|-----
  1 | [=>                                ] | 5% | 9400 KB/s | 940 KB
  2 | [====>                            ] | 15% | 15720 KB/s | 2 MB
  3 | [=====>                        ] | 25% | 16120 KB/s | 4 MB
  ...
 11 | [=====] | 100% | 7720 KB/s | 16 MB
→ 10 ticks (1000 ms), trung bình 16,384 KB/s (16 MB/s)
```

Output mẫu v1 (16 MB) – minh họa sự chậm chạp:

```
  1 | [>                                ] | 0% | 290 KB/s | 29 KB
  2 | [>                                ] | 0% | 485 KB/s | 77 KB
  ...
353 | [=====] | 100% | 240 KB/s | 16 MB
→ 352 ticks (35200 ms), trung bình 465 KB/s
```

→ **v1 cần 353 dòng (35 giây), v8 chỉ cần 11 dòng (1 giây)** – đây là cách trực quan nhất để thấy speedup 35× là thực tế có thể quan sát được.

3 Personal contribution

Member 1 (Nguyễn Sỹ Tuấn): Triển khai v8 (newpipe – tổng hợp 6 cơ chế), viết `pipe_live.c` (real-time throughput monitor), viết `docs/v8.md` và `docs/v1_vs_v8_os_theory.md`, chạy benchmark đầy đủ v1–v8, phân tích OS-theory.

Member 2 (Lê Huy Sơn): Triển khai v6 (priority boost – thay đổi scheduler và `struct proc`), viết `pipe_pingpong.c` (round-trip IPC), viết `pipe_fanout.c` (fan-out pattern), phân tích kết quả scheduling.

Member 3 (Vũ Tiến Long): Triển khai v7 (cache-line alignment), viết `pipe_wc.c` (producer-consumer), viết `pipe_chain.c` (3-stage pipeline), kiểm tra correctness toàn bộ v6–v8 (`usertests`, `pipetest`).

4 Developed code review

4.1 Algorithmic development

Các kỹ thuật chính phát triển trong tuần này:

1. **Priority-based scheduling (v6):** Thêm `priority` vào `struct proc`, `wakeup()` gán `priority = 10`, scheduler chọn process có `priority` cao nhất, `yield()` decay `priority` 1 đơn vị/tick. Tránh starvation vì `priority` về 0 sau tối đa 10 tick.
2. **Cache-line padding (v7):** Chèn `char _pad_w[60]` và `char _pad_r[60]` giữa `nwrite` và `nread` để đặt hai biến lên hai cache line 64-byte khác nhau. Đây là kỹ thuật chuẩn trong Linux kernel (`__cacheline_aligned`).
3. **2-page ring buffer với page-indexed addressing (v8):** `data[DATA_PAGES]` với `DATA_PAGES = 2`, tính `page_idx = pos / PGSIZE` và `page_off = pos % PGSIZE` trước mỗi lần `copyin/copyout` để đảm bảo không vượt ranh giới trang.
4. **OOM-safe rollback (v8):** Loop cấp `DATA_PAGES` lần `kalloc()` – nếu lần thứ i thất bại, free các trang $0..i - 1$ trước khi `goto bad`.
5. **Lazy wakeup với dual flag (v8):** `need_wakeup` (writer side) và `was_full` (reader side) – tránh gọi `wakeup()` khi peer đang chạy bình thường.

4.2 Phân tích so sánh tổng hợp v1–v8

Phiên bản	Cơ chế chính	Cap (B)	Wakeup cost	ctx-sw/4MB	kalloc
v1	byte-by-byte	512	$O(NPROC) \times n$	~16,384	1
v2	bulk copy	512	$O(NPROC) \times \text{ít}$	~8,192	1
v3	chunked ring	2,048	$O(NPROC) \times \text{ít}$	~2,048	1
v4	page-isolated	4,096	$O(NPROC) \times \text{rất ít}$	~1,024	2
v5	lazy wakeup	4,096	$O(1)$ fast path	~1,024	2
v6	priority boost	4,096	$O(1) + \text{boost}$	~1,024	2
v7	cache-line align	4,096	$O(1) + \text{boost}$	~1,024	2
v8	all + 8KB	8,192	$O(1) + \text{boost}$	~512	3

4.3 Source code enhanced and/or revised

`kernel/proc.h` – thêm `priority` field và constants:

```
#if PIPE_VERSION >= 6
#define PRIORITY_BASE 0
#define PRIORITY_BOOST 10
// trong struct proc:
int priority;
#endif
```

`kernel/proc.c` – scheduler với `priority` selection và `yield()` decay:

```
// Chọn process RUNNABLE có priority cao nhất
struct proc *chosen = 0;
for(p = proc; p < &proc[NPROC]; p++){
    if(p->state == RUNNABLE){
        if(chosen == 0 || p->priority > chosen->priority)
            chosen = p;
    }
}

// yield() -- giảm priority mỗi tick
void yield(void) {
    struct proc *p = myproc();
    acquire(&p->lock);
    if(p->priority > PRIORITY_BASE)
        p->priority--;
    p->state = RUNNABLE;
```

```

    sched();
    release(&p->lock);
}

```

kernel/pipe.c – v8 struct và pipewrite core:

```

#ifdef PIPE_VERSION == 8
#define DATA_PAGES    2
#define PIPE_TOTAL     (DATA_PAGES * PGSIZE)

struct pipe {
    struct spinlock lock;
    char  *data[DATA_PAGES]; // 2 trang vat ly rieng biet
    uint  nwrite;
    char  _pad_w[60];        // cache-line padding
    uint  nread;
    char  _pad_r[60];
    int    need_wakeup;      // lazy wakeup: reader dang ngu?
    int    was_full;         // lazy wakeup: writer dang ngu?
    int    readopen, writeopen;
};

// Trong pipewrite -- page-indexed copyin:
uint pos      = pi->nwrite % PIPE_TOTAL;
uint page_idx = pos / PGSIZE;
uint page_off = pos % PGSIZE;
uint till_page = PGSIZE - page_off;
int  to_copy   = umin(n - i, umin(avail, till_page));
copyin(pr->pagetable, pi->data[page_idx] + page_off, addr + i, to_copy);
pi->nwrite += to_copy;
#endif

```

4.4 Testing result

usertests và pipetest – v6, v7, v8 đều pass:

```

$ usertests -q
...
ALL TESTS PASSED

$ pipetest
pipe basic:      OK
pipe large write: OK
pipe blocking:   OK
pipe close:      OK
ALL PIPE TESTS PASSED

```

Benchmark tổng hợp v1–v8 (chunk_bytes = 1024, total_bytes = 4 MB):

version	throughput_bps	KB/s	speedup vs v1
v1	551,882	539	1.00x
v2	1,582,756	1,546	2.87x
v3	3,355,443	3,277	6.08x
v4	3,994,575	3,901	7.24x
v5	6,990,506	6,827	12.67x
v6	8,388,608	8,192	15.20x
v7	8,388,608	8,192	15.20x
v8	13,981,013	13,653	25.33x

4.5 Đánh giá tổng thể dự án – Amdahl’s Law

Định luật Amdahl – phân tích speedup thực tế:

$$\text{Speedup}(v8) = \frac{1}{(1-p) + p/s}$$

Trong đó $p \approx 0.97$ là phần chi phí có thể tối ưu (từ cost model), s là speedup của phần đó.

Đo thực tế: $\text{Speedup}(v8/v1) = 36.5 \times$ tại `chunk_bytes = 4096`.

Dự đoán cost model: $\sim 23 \times$. Thực tế vượt dự đoán vì priority boost (v6) tác động mạnh hơn mô hình tính ước tính – scheduler delay trong v1 lớn hơn dự kiến.

Chuỗi tối ưu và contribution từng bước:

v1 → v2: bulk copy	→ 3.2x	(loại bỏ per-byte copyin)
v2 → v3: ring buffer	→ 3.5x	(tăng capacity + no wrap)
v3 → v4: page isolation	→ 1.6x	(cache locality + 4KB cap)
v4 → v5: lazy wakeup	→ 1.0x	(fast path, ít sleep)
v5 → v6: priority boost	→ 1.3x	(giảm scheduler delay)
v6 → v7: cache-line align	→ 0.9x	(giảm trên single-CPU)
v7 → v8: 8KB buffer + all	→ 1.7x	(capacity 2x, tổng hợp)

v1 → v8: TONG	→ 36.5x	(do thực tế)

5 TODO

(a) List of remaining issues:

- v7 (cache-line alignment) hiển thị speedup âm (-14%) trên single-CPU vì struct lớn hơn. Cần benchmark lại với `CPUS=2` để xác nhận lợi ích thực trong môi trường multi-core.
- `pipe_live.c` chỉ đo throughput từ phía reader. Có thể mở rộng thành 3 tiến trình để hiển thị cả hai phía đồng thời.
- Benchmark chưa đo **latency** khi `total_bytes` nhỏ (< 1 KB) – quan trọng cho workload RPC thực tế.
- Chưa thử nghiệm v8 với nhiều pipe đồng thời (stress test OOM path cho `pipealloc` 3 lần `kalloc()`).

(b) List of tasks to carry out in the next period: Dự án đã hoàn thành **100%** mục tiêu tối ưu hóa (v1–v8). Các công việc tiếp theo mang tính tài liệu hóa và trình bày:

- Chuẩn bị **slide tổng kết** – bottleneck map, chuỗi speedup v1→v8, kết luận Amdahl.
- Viết **báo cáo cuối kỳ** tổng hợp toàn bộ 8 tuần, bao gồm kết quả benchmark đầy đủ và phân tích OS-theory từ `docs/v1_vs_v8_os_theory.md`.
- Quay **video demo** chạy `pipe_live` trong QEMU với v1 và v8 side-by-side để minh họa trực quan.

(c) Milestone:

- Hoàn thành:** Toàn bộ 8 phiên bản (v1–v8) đã triển khai, benchmark, và tài liệu hóa.
- Kết quả chính:** v8 đạt throughput **20.97 MB/s** (`chunk=4096`), speedup **36.5x** so với v1 baseline (574 KB/s).

- **Sẵn sàng:** Tài liệu phân tích OS-theory (`docs/v8.md`, `docs/v1_vs_v8_os_theory.md`), biểu đồ so sánh (`results/compare_throughput.png`), và 5 chương trình IPC use-case minh họa thực tế.

6 Notes for revision

- Tất cả benchmark chạy trên QEMU `CPUS=1`. v7 (cache-line alignment) cần đo lại với `CPUS=2` để thấy lợi ích thực – trên single-CPU struct lớn hơn làm hại nhỏ, nhưng trên multi-core loại bỏ false sharing là cải thiện quyết định.
- Speedup v8 ($36.5\times$) vượt dự đoán cost model ($\sim 23\times$): nguyên nhân chính là priority boost tác động mạnh hơn dự đoán tính, do scheduler delay trong v1 lớn hơn khi nhiều process cạnh tranh.
- `pipe_pingpong` speedup ($\sim 6\times$) thấp hơn các benchmark streaming ($\sim 27\text{--}57\times$) là đúng về lý thuyết: RTT bị giới hạn bởi số lần wakeup cố định (2 lần/round), không thể tránh bằng buffer lớn hơn; chỉ priority boost giảm thời gian chờ mỗi hop.
- v8 dùng 3 lần `kalloc()` mỗi pipe (1 trang struct + 2 trang data). Cần ghi rõ trong tài liệu cuối kỳ: hệ thống chỉ có 128 MB RAM (QEMU `-m 128M`), tổng $\sim 32K$ trang, mỗi pipe chiếm 3 trang $\rightarrow$ tối đa $\sim 10K$ pipe đồng thời – đủ cho mọi workload thực tế trong xv6.